

## Monitores

## Ventajas

- Simples.
- Eficientes (no hay busy-waiting).

## Desventajas (Muy bajo nivel)

- Si olvidamos el signal: **deadlock**.
- Si olvidamos el wait: no garantizamos exclusión.
- La sincronización no está vinculada a los datos.
  - Las instrucciones encargadas de la sincronización pueden aparecer en cualquier parte.

# Monitores

- Combina tipo de datos abstractos y exclusión mutua.
  - Propuesto por **Tony Hoare [1974]**.
- Empleados masivamente en lenguajes modernos:
  - Pthreads
  - Java
  - C#

# Monitores : ADT + Sincronización

## Definición y Encapsulamiento

- La responsabilidad sincronizar correctamente está alocada en **módulos aislados**.
- Un monitor para cada objeto o grupo de objetos relacionados (tipo de datos).
- Todos los campos de datos de un monitor son **privados**.

## Exclusión Mutua y Concurrency

- **Mismo monitor:** La abstracción garantiza que las operaciones se ejecutan en exclusión mutua.
- **Distintos monitores:** Las ejecuciones de sus operaciones pueden entrelazarse libremente.

## Generalización de la POO

- Es una evolución de las clases de la Programación Orientada a Objetos.
- Añade la restricción de que **solo un proceso a la vez** puede operar sobre un objeto.

# Ejemplo

## Example

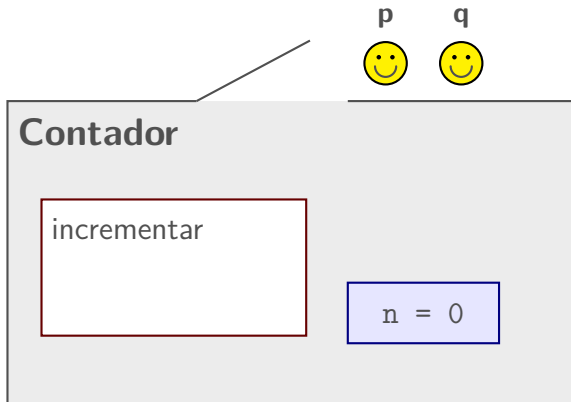
```
Monitor Contador {  
    private int contador= 0  
    public void incrementar(){  
        contador++  
    }  
}
```

```
thread p  
p1: CS.incrementar()
```

```
thread q  
q1: CS.incrementar()
```

- **Encapsulamiento:** La variable contador es privada y no se puede acceder desde afuera del monitor CS.
- **Exclusión mutua:** Solo un proceso a la vez puede ejecutar una operación del monitor.
- **Atomicidad garantizada:** Aunque incrementar tiene varias instrucciones, los procesos p y q no pueden entrelazarse.
- **Resultado:** Se garantiza que el valor final de contador será 2.

# Simulación Gráfica de un Monitor



## Estado Inicial

- El monitor está vacío y la puerta abierta.
- Los procesos  $p$  y  $q$  quieren entrar.
- La elección depende de la implementación. Por ahora asumimos **no determinismo**.

# Simulación Gráfica de un Monitor



## Contador

incrementar



p

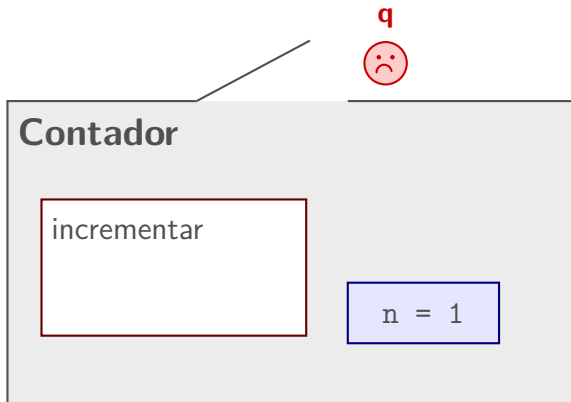
n = 0

## Exclusión Mutua

- $p$  es elegido e ingresa a incrementar.
- La puerta se cierra automáticamente.
- $q$  queda bloqueado fuera.



# Simulación Gráfica de un Monitor



## Modificación de $n$

- $p$  termina su bloque de operaciones.
- El valor de la variable cambia a  $n = 1$ .
- El lock se abre para el siguiente proceso.

# Simulación Gráfica de un Monitor

## Contador

incrementar



$q$

$n = 2$

## Resultado Final

- $p$  se retira.  $q$  toma su turno de manera exclusiva.
- Al finalizar  $q$ , el valor definitivo es  $n = 2$ .
- **Atomicidad total garantizada.**

## Variables de Condición

- El monitor garantiza **exclusión mutua** implícita sobre las variables privadas.
- Muchos problemas requieren sincronización adicional y explícita.

# Variables de Condición

- El monitor garantiza **exclusión mutua** implícita sobre las variables privadas.
- Muchos problemas requieren sincronización adicional y explícita.
- *Ejemplo clásico:* **Productor-Consumidor**.
  - El productor se bloquea si el buffer está **lleno**.
  - El consumidor se bloquea si el buffer está **vacío**.

# Variables de Condición

- El monitor garantiza **exclusión mutua** implícita sobre las variables privadas.
- Muchos problemas requieren sincronización adicional y explícita.
- *Ejemplo clásico: Productor-Consumidor.*
  - El productor se bloquea si el buffer está **lleno**.
  - El consumidor se bloquea si el buffer está **vacío**.

## Variables de condición (o condición)

Son colas de espera explícitas (*FIFO*) manejadas por la abstracción que permiten definir sincronizaciones adicionales entre operaciones.

- No almacenan valores; representan procesos temporalmente suspendidos.
- Permiten que un proceso libere el monitor voluntariamente cuando un recurso no está disponible.

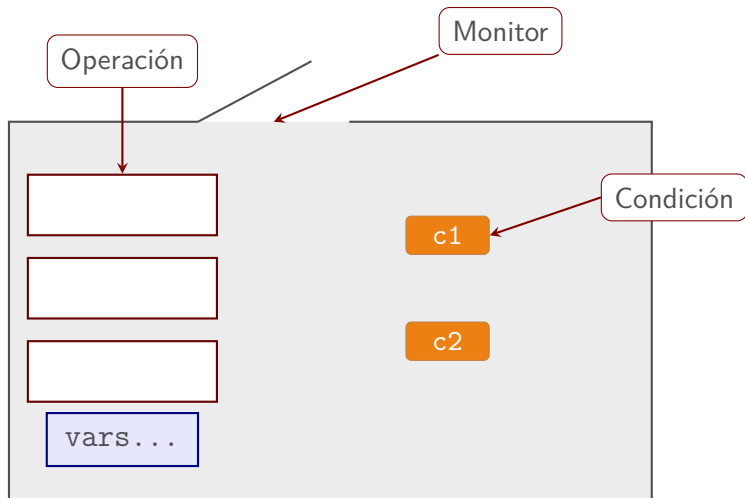
## Elementos principales de un monitor

- Un **conjunto de operaciones** (procedimientos) encapsuladas en módulos (o clases).
- Existe un único **lock** (mutex) global que asegura la exclusión mutua entre todas las operaciones en el monitor.
  - **Exclusión mutua automática.**
- **Variables de condiciones** (*condition variables*) para definir sincronización condicional.

# Variables de condición

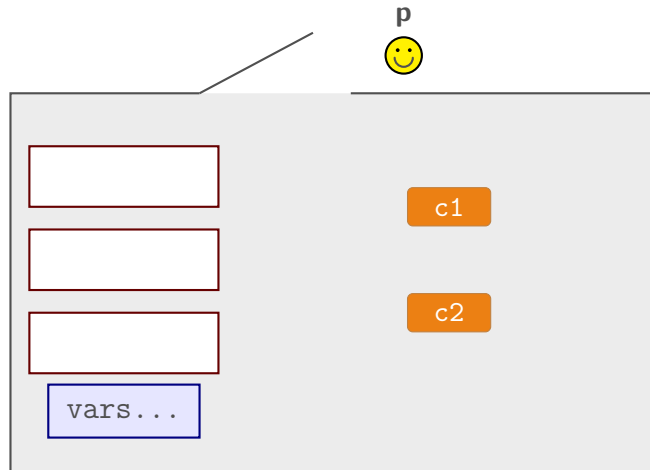
- Son globales al monitor.
- Cada una representa **una cola de procesos bloqueados**.
- Poseen **dos operaciones principales**:
  - **wait(c)**: Bloquea el proceso en ejecución y lo asocia a la variable *c*. El proceso **debe liberar el lock** de exclusión mutua.
  - **signal(c)**: Desbloquea al primer proceso bloqueado en la variable *c*.

# Estructura de Sincronización en Monitores





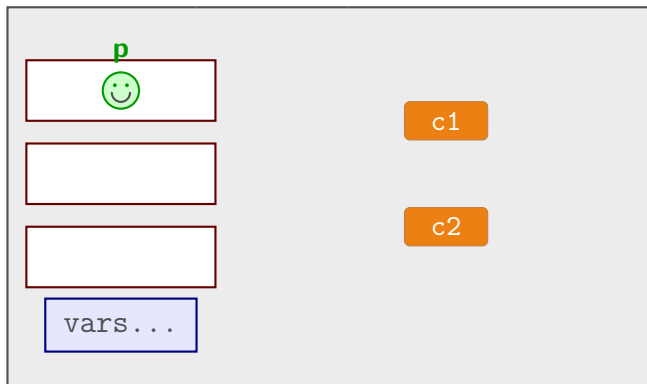
# Comportamiento Típico en Monitores



## 1. Intento de Acceso

- El proceso  $p$  llega al monitor queriendo ejecutar una operación.
- Al encontrar la puerta abierta, se dispone a ingresar.

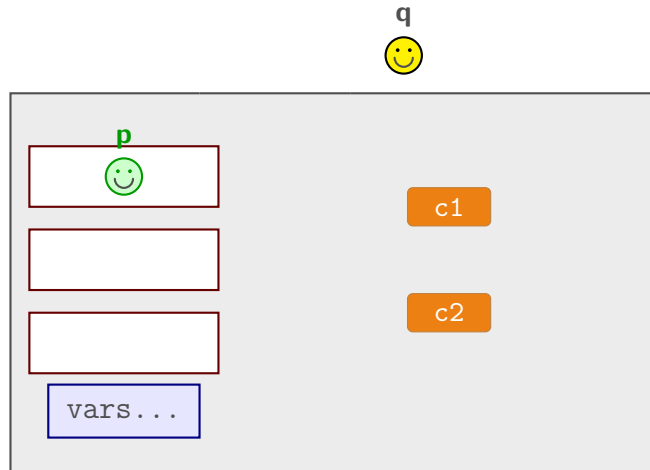
# Comportamiento Típico en Monitores



## 2. Exclusión Mutua

- $p$  ingresa con éxito a la primera operación.
- El lock se activa y la puerta **se cierra**.
- Ningún otro proceso puede irrumpir temporalmente.

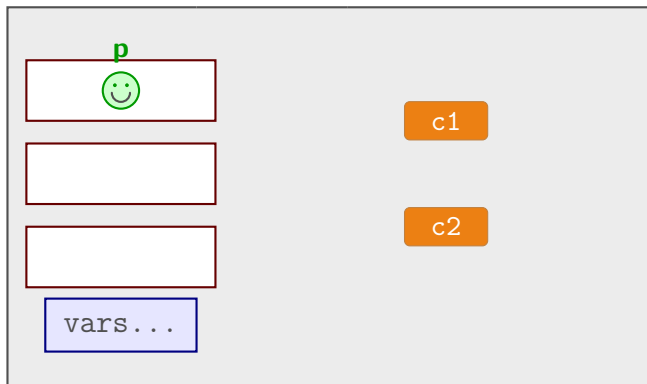
# Comportamiento Típico en Monitores



## 3. Competencia

- Mientras  $p$  continúa activo adentro, arriba un proceso  $q$ .
- Como el monitor está ocupado,  $q$  debe esperar en la entrada.

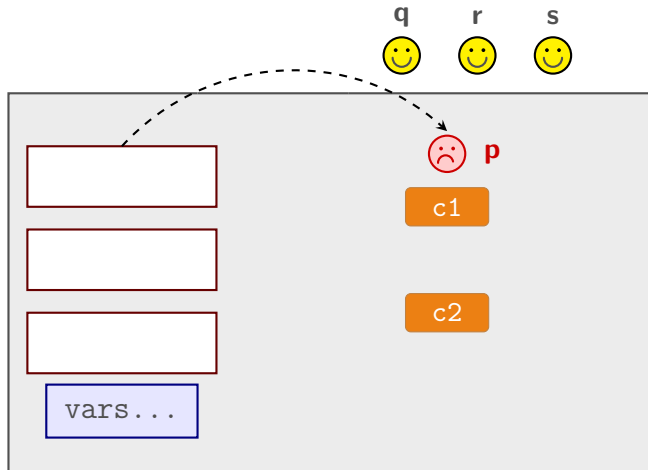
# Comportamiento Típico en Monitores



## 4. Llegan más procesos

- Llegan más hilos concurrentes ( $r, s$ ).
- Debido a la exclusión mutua, se acumulan esperando su turno en la entrada.

# Comportamiento Típico en Monitores



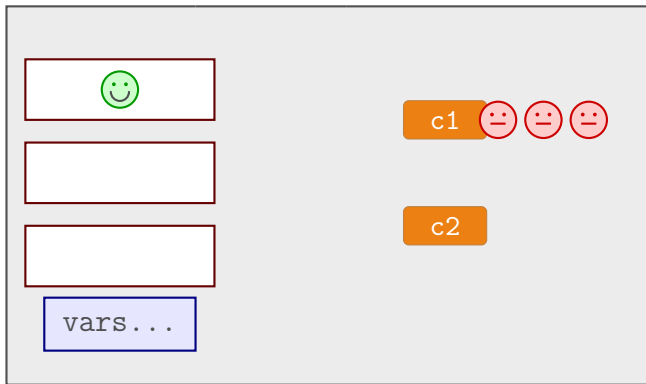
## 5. Bloqueo en Condición

- *p* se bloquea al invocar un `wait(c1)`.
- Abandona la exclusión: se agrega a la cola de *c1* y la puerta **se abre** automáticamente.

## Operación wait(c)

- Similar a la operación del wait del semáforo.
  - wait(c) bloquea el proceso en ejecución y lo asocia a la variable c.
- El proceso bloqueado debe **liberar el lock** de exclusión mutua.
  - Permite que ingrese algún otro proceso al monitor.
- **¿En qué difiere esta operación de la operación wait de semáforo?**

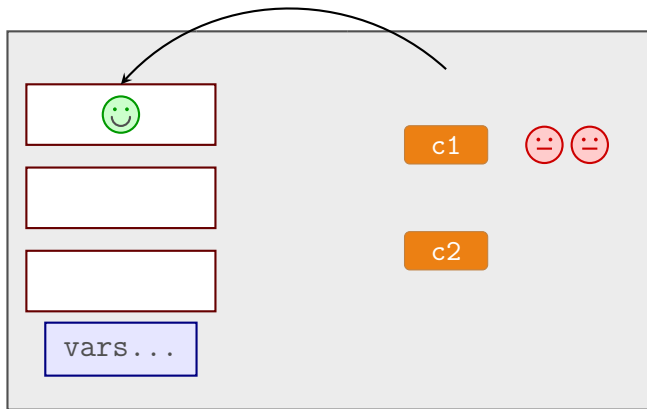
## Comportamiento Típico: Operación `signal(c1)` al final



### 6. Estado de bloqueo

- En una variable de condición `c1` se encuentra una cola de hilos suspendidos esperando por un recurso o estado.
- Un proceso en ejecución modifica el estado (o genera el recurso) que están esperando los procesos bloqueados.

## Comportamiento Típico: Operación `signal(c1)` al final



### 7. Operación Signal

- El proceso que está adentro ejecuta la instrucción `signal(c1)` como **última instrucción antes de salir**.
- La llamada a `signal` **desbloquea a un proceso** esperando en la cola.
- El proceso se despierta y continúa su ejecución.



## Operación `signal` como última instrucción

- **Desbloquea al primer proceso** bloqueado en la variable de condición.
- Si no hay procesos esperando en la cola, la operación no tiene efecto (no se almacena el `signal`).

## Operación `signal` como última instrucción

- **Desbloquea al primer proceso** bloqueado en la variable de condición.
- Si no hay procesos esperando en la cola, la operación no tiene efecto (no se almacena el `signal`).
- Al desbloquear un proceso, el desbloqueado **continúa su ejecución** desde la instrucción que sigue a la llamada del `wait` que lo bloqueó.

## Operación `signal` como última instrucción

- **Desbloquea al primer proceso** bloqueado en la variable de condición.
- Si no hay procesos esperando en la cola, la operación no tiene efecto (no se almacena el `signal`).
- Al desbloquear un proceso, el desbloqueado **continúa su ejecución** desde la instrucción que sigue a la llamada del `wait` que lo bloqueó.
- **¿Qué sucede con el lock?**  
El proceso que ejecuta la llamada a `signal(c)` **termina** y **pasa el lock** de exclusión mutua del monitor al proceso que se desbloquea.

# Implementación de las Variables de Condición

## Estructura y atomicidad

- Una **cola FIFO** de procesos bloqueados.
- Las operaciones se ejecutan de manera atómica.

# Implementación de las Variables de Condición

## Estructura y atomicidad

- Una **cola FIFO** de procesos bloqueados.
- Las operaciones se ejecutan de manera atómica.

```
waitC(cond)
  append p to cond
  p.state = blocked
  monitor.lock.unlock()
```

- **Suspensión:** El proceso que ejecuta wait se suspende inmediatamente en la cola cond.
- **Libera el Lock** para permitir que otro proceso ingrese al monitor.

# Implementación de las Variables de Condición

## Estructura y atomicidad

- Una **cola FIFO** de procesos bloqueados.
- Las operaciones se ejecutan de manera atómica.

```
signalC(cond)
  if cond != empty
    q = pop cond
    q.state = ready
```

- **Cambio de Estado:** El proceso que se despierta pasa al estado **listo** (*ready*).
- **Riesgo de violación mutex:** si `signal` no es la última operación del proceso dentro del monitor, se puede violar la exclusión mutua.

# Implementación de las Variables de Condición

## Estructura y atomicidad

- Una **cola FIFO** de procesos bloqueados.
- Las operaciones se ejecutan de manera atómica.

```
empty(cond)  
    return cond == empty
```

- Devuelve un valor booleano indicando si la cola de procesos bloqueados está vacía o no.

# Simulación de un Semáforo mediante Monitores



# Simulación de un Semáforo mediante Monitores

## Estructura del Monitor:

- **sv:** Variable entera privada que representa el valor actual del semáforo.
- **noCero:** Variable de condición donde se suspenden los procesos cuando el semáforo llega a cero.

# Simulación de un Semáforo mediante Monitores

## Estructura del Monitor:

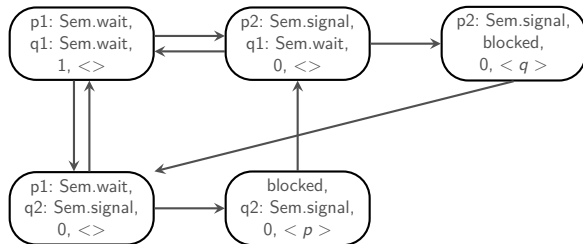
- **sv:** Variable entera privada que representa el valor actual del semáforo.
- **noCero:** Variable de condición donde se suspenden los procesos cuando el semáforo llega a cero.

```
monitor Semaforo {  
    private int sv;  
    private Condition noCero;  
  
    public Semaforo(int v) {  
        //pre: v >= 0  
        this.sv = v;  
    }  
}
```

# Operaciones del Monitor Semáforo

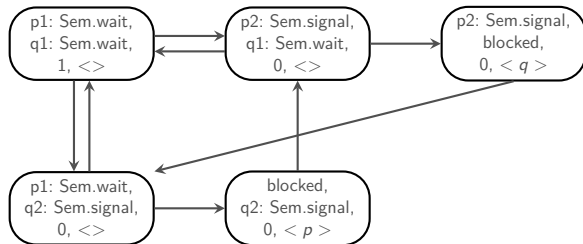
```
public void wait() {  
    if (sv == 0)  
        wait(noCero);  
  
    sv--;  
}  
  
public void signal() {  
    sv++;  
    signal(noCero);  
}  
} // fin monitor
```

# Corrección de la simulación del Semáforo



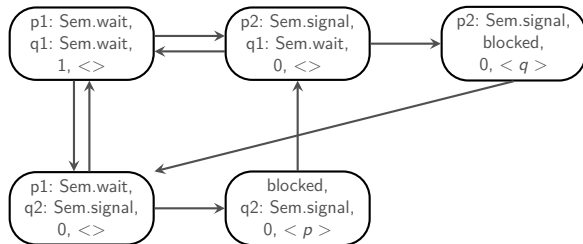
- **Paso Único Atómico:** Las operaciones se ejecutan en exclusión mutua. Sus instrucciones se agrupan en una única transición.

## Corrección de la simulación del Semáforo



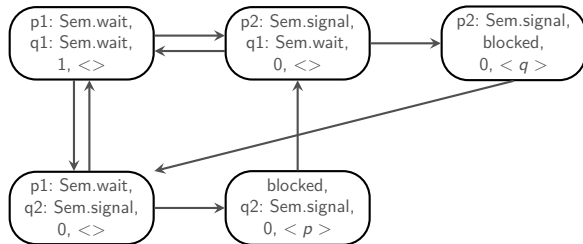
- **Simplicidad:** El diagrama es compacto porque las transiciones internas de las sentencias del monitor se abstraen en un solo paso.

# Corrección de la simulación del Semáforo



- **Bloqueo en cola:** Cuando un proceso ejecuta `Sem.wait` cuando  $s = 0$ , se bloquea en la cola `noCero`.

# Corrección de la simulación del Semáforo



- **Garantía:** No existe un estado de la forma (p2: Sem.signal, q2: Sem.signal), hay **exclusión mutua**.

## Implementación de un Buffer (dimensión 1) con Monitores



# Implementación de un Buffer (dimensión 1) con Monitores

## Variables de Condición:

- **hayEspacio:** Los productores esperan en esta cola hasta que el buffer se libere.
- **conDato:** Los consumidores esperan en esta cola hasta que haya un dato disponible.

## Estructura del Monitor:

```
monitor Buffer {  
    private condition hayEspacio;  
    private condition conDato;  
    private boolean estaVacio = true;  
    private int buf;  
  
    // ... sigue en la siguiente diapositiva
```

# Operaciones del Monitor Buffer

## Consumidor (leer)

```
public int leer() {  
    if (estaVacio)  
        wait(conDato);  
  
    estaVacio = true;  
    int res = buf;  
    signal(hayEspacio);  
    return res;  
}
```

# Operaciones del Monitor Buffer

## Consumidor (leer)

```
public int leer() {  
    if (estaVacio)  
        wait(conDato);  
  
    estaVacio = true;  
    int res = buf;  
    signal(hayEspacio);  
    return res;  
}
```

## Productor (escribir)

```
public void escribir(int dato) {  
    if (!estaVacio)  
        wait(hayEspacio);  
  
    buf = dato;  
    estaVacio = false;  
    signal(conDato);  
}
```

# Operaciones del Monitor Buffer

## Consumidor (leer)

```
public int leer() {  
    if (estaVacio)  
        wait(conDato);  
  
    estaVacio = true;  
    int res = buf;  
    signal(hayEspacio);  
    return res;  
}
```

## Productor (escribir)

```
public void escribir(int dato) {  
    if (!estaVacio)  
        wait(hayEspacio);  
  
    buf = dato;  
    estaVacio = false;  
    signal(conDato);  
}
```

Notar: que en leer, signal no es la última instrucción. Se puede solapar el return con un escribir. Aquí no habría problemas porque no se tocan las variables compartidas

# Disciplinas para signal

- Hasta ahora consideramos la versión en la cual el `signal` implica la **salida del monitor**:
  - El `signal` se encuentra estrictamente al final del método.
  - El lock mutex se transfiere de inmediato al proceso que se despierta.
- **Otras posibilidades y variantes:**
  - ¿Qué sucede si el `signal` **no está al final** del procedimiento?

# Coexistencia tras un Signal

- **Violación de mutex:**

- El proceso **p** ejecuta `signalC(cond)` y puede continuar con su siguiente instrucción.
  - El proceso **q** (desbloqueado) puede continuar desde la instrucción posterior a su `waitC(cond)`.
- Este escenario debería ser posible. La especificación de un monitor exige que **como máximo un proceso a la vez** pueda estar ejecutando instrucciones adentro.

# Coexistencia tras un Signal

- **Violación de mutex:**

- El proceso **p** ejecuta `signalC(cond)` y puede continuar con su siguiente instrucción.
- El proceso **q** (desbloqueado) puede continuar desde la instrucción posterior a su `waitC(cond)`.

- Este escenario debería ser posible. La especificación de un monitor exige que **como máximo un proceso a la vez** pueda estar ejecutando instrucciones adentro.
- **Necesidad de una Disciplina:** Se debe evitar que haya dos procesos al mismo tiempo.

# Coexistencia tras un Signal

- **Violación de mutex:**

- El proceso **p** ejecuta `signalC(cond)` y puede continuar con su siguiente instrucción.
- El proceso **q** (desbloqueado) puede continuar desde la instrucción posterior a su `waitC(cond)`.

- Este escenario debería ser posible. La especificación de un monitor exige que **como máximo un proceso a la vez** pueda estar ejecutando instrucciones adentro.

- **Necesidad de una Disciplina:** Se debe evitar que haya dos procesos al mismo tiempo.

- **E (Entry):** Procesos bloqueados intentando entrar por primera vez al monitor.
- **W (Waiting):** Procesos recién liberados de una cola de condición.
- **S (Signaling):** Procesos que acaban de ejecutar una llamada a `signal`.



# El Requerimiento de Reanudación Inmediata (IRR)

- **Restricciones de Sentido Común:** Existen 13 formas de ordenar las prioridades ( $E, W, S$ ). Muchas carecen de sentido práctico.
  - Hacer que  $E > W$  o  $E > S$  provocaría **inanición**.

# El Requerimiento de Reanudación Inmediata (IRR)

- **Restricciones de Sentido Común:** Existen 13 formas de ordenar las prioridades ( $E, W, S$ ). Muchas carecen de sentido práctico.
  - Hacer que  $E > W$  o  $E > S$  provocaría **inanición**.
- **La Política Clásica de Hoare ( $E < S < W$ ):**
  - Conocida formalmente como **Immediate Resumption Requirement (IRR)** o disciplina de *Signal y Espera*.
  - Cuando un proceso bloqueado se despierta, toma la CPU e **inicia su ejecución de inmediato**.

# El Requerimiento de Reanudación Inmediata (IRR)

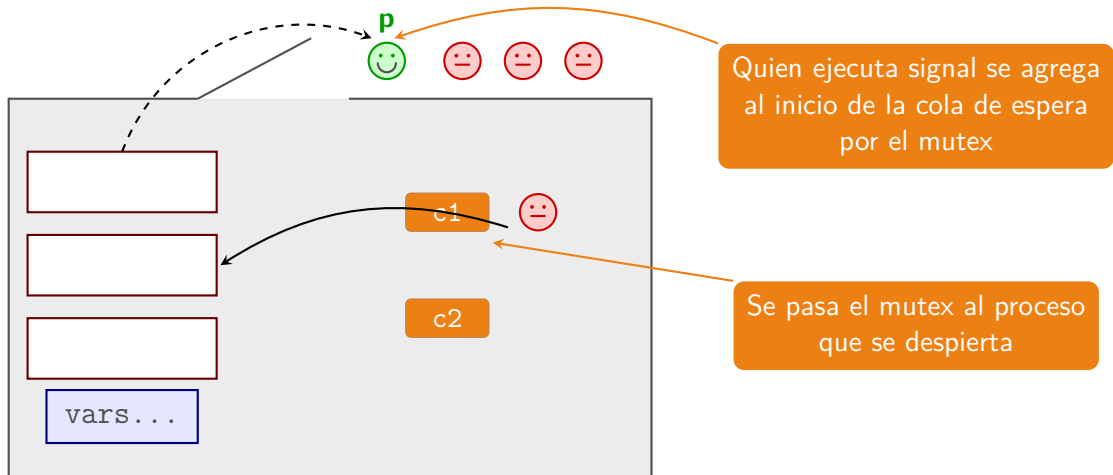
- **Restricciones de Sentido Común:** Existen 13 formas de ordenar las prioridades ( $E, W, S$ ). Muchas carecen de sentido práctico.
  - Hacer que  $E > W$  o  $E > S$  provocaría **inanición**.
- **La Política Clásica de Hoare ( $E < S < W$ ):**
  - Conocida formalmente como **Immediate Resumption Requirement (IRR)** o disciplina de *Signal y Espera*.
  - Cuando un proceso bloqueado se despierta, toma la CPU e **inicia su ejecución de inmediato**.
- **Intuición:** El proceso que hace `signal` acaba de modificar el estado para que la condición se cumpla. Si el hilo despertado entra de forma inmediata y atómica, tiene la certeza de que la condición **sigue siendo verdadera** y puede avanzar sin reevaluar.

## Jerarquías de Prioridades: Hoare vs. Java (Mesa)

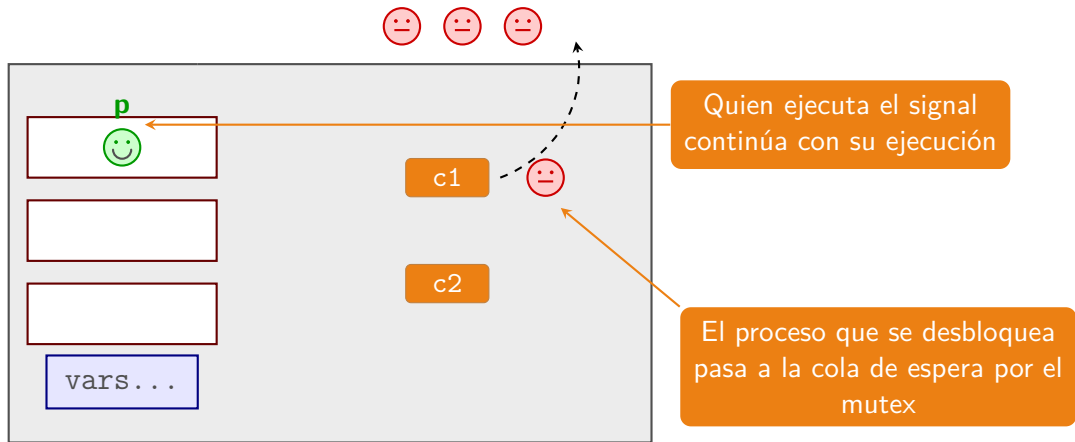
- Las distintas semánticas de monitores se diferencian por cómo ordenan las prioridades:

| Semántica                               | Orden       | Mutex                                                                                                                                                     |
|-----------------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Monitor Clásico</b><br>(Hoare / IRR) | $E < S < W$ | El proceso despertado ( $W$ ) adquiere el mutex. El señalizador ( $S$ ) va primero en la <b>cola</b> del mutex. <b>El mutex es fair</b>                   |
| <b>Monitor Moderno</b><br>(Mesa/Java)   | $E = W < S$ | El señalizador ( $S$ ) termina primero. El despertado ( $W$ ) compite en igualdad con las nuevas entradas ( $E$ ). <b>El mutex es generalmente unfair</b> |

# Signal y Espera Urgente (Hoare)



# Signal y Continúa (Mesa)



## Política

- El proceso **p** que ejecuta `signal` continúa su ejecución de forma normal dentro del monitor, mientras que:
- El proceso **q** que se desbloquea abandona la variable de condición y **se une a la cola de espera** por el mutex de entrada.

## Política

- El proceso **p** que ejecuta `signal` continúa su ejecución de forma normal dentro del monitor, mientras que:
- El proceso **q** que se desbloquea abandona la variable de condición y **se une a la cola de espera** por el mutex de entrada.

## Problema crítico

- El proceso **p** puede seguir modificando el estado y las variables globales del monitor **luego de haber ejecutado el `signal`**.



## Política

- El proceso **p** que ejecuta `signal` continúa su ejecución de forma normal dentro del monitor, mientras que:
- El proceso **q** que se desbloquea abandona la variable de condición y **se une a la cola de espera** por el mutex de entrada.

## Problema crítico

- El proceso **p** puede seguir modificando el estado y las variables globales del monitor **luego de haber ejecutado el `signal`**.
- Cuando **q** finalmente logre volver a entrar al monitor, la condición por la que se despertó podría haber dejado de ser verdadera.

## Signal y continúa: Patrón de Programación

- Debido a que la condición puede cambiar antes de recuperar el mutex, **necesitamos programar teniendo en cuenta esto.**

# Signal y continúa: Patrón de Programación

- Debido a que la condición puede cambiar antes de recuperar el mutex, **necesitamos programar teniendo en cuenta esto.**

```
public void wait() {  
    // Al despertar,  
    vuelve a verificar  
    while (sv == 0)  
        wait(noCero);  
  
    sv--;  
}
```

- Se debe utilizar un patrón conocido como “**pasar la condición**”:
  - Se sustituye if por un bucle while para reevaluar.

# Signal y continúa: Patrón de Programación

- Debido a que la condición puede cambiar antes de recuperar el mutex, **necesitamos programar teniendo en cuenta esto.**

```
public void wait() {  
    // Al despertar,  
    vuelve a verificar  
    while (sv == 0)  
        wait(noCero);  
  
    sv--;  
}
```

- Se debe utilizar un patrón conocido como “**pasar la condición**”:
  - Se sustituye if por un bucle while para reevaluar.
- **Consecuencia:** Riesgo de **inanición** si procesos nuevos le quitan el mutex al recién desbloqueado.

## Signal y continúa: la preferida hoy

- **Percepción inicial:** Parece menos intuitivo debido a la necesidad de reevaluar las condiciones en un bucle `while`.
- Es la **disciplina preferida hoy** en la mayoría de los lenguajes de programación s por los siguientes motivos:
  - **Semántica más simple:** No requiere de un traspaso atómico del lock entre procesos.
  - **Compatibilidad con políticas de scheduling:** por ejemplo políticas de prioridad.

## Signal y continúa: Equidad (Fairness)

```
public void wait() {  
    while (sv == 0)  
        wait(noCero);  
  
    sv--;  
}  
  
public void signal() {  
    sv++;  
    signal(noCero);  
}
```

¿Es fair? ¿Qué sucede con un proceso que estaba esperando para adquirir el lock?

# Signal y continúa: Falta de Equidad

- **Problema de Fairness:** No es fair, dado que un proceso fuera del monitor podría adelantarse y **robar el token** (el recurso) antes de que el proceso despertado logre reingresar.
- **Alternativa de solución:**
  - El proceso que ejecuta `signal` le pasa directamente la información de que `sv` tiene un valor positivo al proceso que se despierta.
  - Para implementar esto de forma segura, necesitamos utilizar la operación `empty(cv)`.
  - Esta función nos permite **conocer si existen procesos bloqueados** en la variable de condición.

# Semáforo Equitativo con empty()

## Solución al robo del Token:

- Se cambia el while por un if bajo una nueva condición.
- El proceso que hace signal **no incrementa** sv si detecta que hay alguien esperando en la cola.
- El token se transfiere directamente de forma lógica al proceso despertado, impidiendo que hilos externos se lo quiten.

```
public void wait() {  
    if (sv == 0)  
        wait(noCero);  
    else  
        sv--;  
}
```

```
public void signal() {  
    if (empty(noCero))  
        sv++;  
    else  
        signal(noCero);  
}
```



# Buffer de Capacidad $N$ (Signal y Salida Urgente)

## Estructura del Monitor:

- Utiliza un arreglo circular de tamaño  $N$  y un contador explícito (cant).
- Al ser la semántica de Hoare, las validaciones de las colas de condición se realizan de forma segura mediante un condicional if.

```
monitor Buffer {  
    private condition hayEspacio;  
    private condition conDato;  
    private int buffer[N];  
    private int inicio = 0, fin = 0, cant = 0;  
  
    // ... las operaciones siguen en la proxima  
    diapositiva
```

# Buffer de Capacidad $N$ (Signal y Salida Urgente)

## Productor (escribir)

```
public void escribir(int dato) {  
    if (cant == N)  
        wait(hayEspacio);  
  
    buffer[fin] = dato;  
    fin = (fin + 1) % N;  
    cant++;  
  
    signal(conDato);  
    return;  
}
```

## Consumidor (leer)

```
public int leer() {  
    if (cant == 0)  
        wait(conDato);  
  
    int aux = buffer[inicio];  
    inicio = (inicio + 1) % N;  
    cant--;  
  
    signal(hayEspacio);  
    return aux;  
}
```

# Buffer de Capacidad $N$ (Signal y Continúa)

## Estructura del Monitor:

- Mantiene la misma lógica de arreglo circular y contador explícito (cant).
- **Diferencia clave:** Al utilizar la política de Mesa, es obligatorio proteger las esperas con un bucle while para reevaluar la condición al despertar.

```
monitor Buffer {  
    private condition hayEspacio;  
    private condition conDato;  
    private int buffer[N];  
    private int inicio = 0, fin = 0, cant = 0;  
  
    % ... las operaciones siguen en la proxima  
    diapositiva
```

# Operaciones del Buffer Circular (Mesa)

## Productor (escribir)

```
public void escribir(int dato) {  
    while (cant == N)  
        wait(hayEspacio);  
  
    buffer[fin] = dato;  
    fin = (fin + 1) % N;  
    cant++;  
  
    signal(conDato);  
    return;  
}
```

## Consumidor (leer)

```
public int leer() {  
    while (cant == 0)  
        wait(conDato);  
  
    int aux = buffer[inicio];  
    inicio = (inicio + 1) % N;  
    cant--;  
  
    signal(hayEspacio);  
    return aux;  
}
```

# Buffer N: Optimización de Señalización

## Productor Optimizado

```
public void escribir(int dato) {  
    while (cant == N)  
        wait(hayEspacio);  
  
    buffer[fin] = dato;  
    fin = (fin + 1) % N;  
    cant++;  
  
    if (cant == 1)  
        signal(conDato);  
  
    return;  
}
```

## Consumidor Optimizado

```
public int leer() {  
    while (cant == 0)  
        wait(conDato);  
  
    int aux = buffer[inicio];  
    inicio = (inicio + 1) % N;  
    cant--;  
  
    if (cant == N - 1)  
        signal(hayEspacio);  
  
    return aux;  
}
```

## ¡Error: Lost Wake-up!

- **La intención de la optimización:** Evitar ejecutar `signal()` de forma innecesaria en cada iteración, enviándolo únicamente cuando el buffer pasa de estar vacío a tener un elemento (`cant == 1`).
- **Por qué se rompe en la Semántica de Mesa:**
  1. Supongamos que el buffer está lleno (`cant = N`) y hay **dos consumidores** esperando en la cola `conDato`.
  2. Un productor inserta un elemento, el contador no es 1, por lo que **no hace signal**.
  3. Debido al entrelazado y a que la notificación condicional se "perdió", un hilo que tenía derecho a despertarse se queda suspendido indefinidamente.
- **Conclusión:** En la política de Mesa (Signal & Continua), condicionar el `signal` a un valor puntual suele provocar que los hilos se queden dormidos para siempre. **Se debe señalar siempre.**

# Filósofos Comensales con Monitores

- El monitor encapsula hace atómica la operación de tomar dos tenedores:

```
monitor DiningPhilosophers {  
    // forks[i] cuenta tenedores  
    // libres para el filosofo i  
    private int forks[5] = {2,2,2,2,2};  
    private condition okToEat[5];  
  
    // ... sigue en la proxima slide  
}
```

# Operaciones del Monitor de los Filósofos

## Entrada (takeForks)

```
public void takeForks(int i)
{
    while (forks[i] < 2)
        wait(okToEat[i]);

    // Decrementa los
    // tenedores
    // disponibles de sus
    // vecinos
    forks[left(i)]--;
    forks[right(i)]--;
}
```

## Salida (releaseForks)

```
public void releaseForks(int i)
{
    forks[left(i)]++;
    forks[right(i)]++;

    // Si un vecino ahora tiene
    // 2
    // tenedores, lo despierta
    if (forks[left(i)] == 2)
        signal(okToEat[left(i)]);
    if (forks[right(i)] == 2)
        signal(okToEat[right(i)]);
}
```



## Estructura del Monitor (Semántica de Mesa):

- **writer:** Booleano que indica si hay un escritor activo.
- **readers#:** Contador de lectores que se encuentran leyendo.
- **cond:** Condición única donde esperan tanto lectores como escritores.

```
monitor RW {  
    private condition cond;  
    private boolean writer = false;  
    private int readers = 0;  
  
    // las operaciones siguen en la proxima diapositiva
```

# Operaciones del Monitor RW

## Gestión de Lectores

```
public void startRead() {  
    while (writer)  
        wait(cond);  
    readers++;  
}
```

```
public void endRead() {  
    readers--;  
    if (readers == 0)  
        signalAll(cond);  
}
```

## Gestión de Escritores

```
public void startWrite() {  
    while (writer || readers >  
        0)  
        wait(cond);  
    writer = true;  
}
```

```
public void endWrite() {  
    writer = false;  
    signalAll(cond);  
}
```

# Operaciones del Monitor RW

## Gestión de Lectores

```
public void startRead() {  
    while (writer)  
        wait(cond);  
    readers++;  
}
```

```
public void endRead() {  
    readers--;  
    if (readers == 0)  
        signalAll(cond);  
}
```

## Gestión de Escritores

```
public void startWrite() {  
    while (writer || readers >  
        0)  
        wait(cond);  
    writer = true;  
}
```

```
public void endWrite() {  
    writer = false;  
    signalAll(cond);  
}
```

- **¡Peligro de Lost Wake-up!** Si se reemplaza `signalAll` por un `signal`, una notificación para un escritor podría despertar a un lector (o viceversa).

## Estructura del Monitor (Semántica de Mesa):

- Separar las colas de espera permite despertar selectivamente a lectores o escritores, evitando el uso ineficiente de `signalAll`.

```
monitor RW {  
    private condition readOk;  
    private condition writeOk;  
    private boolean writer = false;  
    private int readers = 0;  
  
    // ... las operaciones siguen en las proximas diapositivas
```

# Operaciones con Colas Separadas

## Operaciones de Lectura

```
public void startRead() {  
    while (writer)  
        wait(readOk);  
    readers++;  
    signal(readOk); // Cascada  
}
```

```
public void endRead() {  
    readers--;  
    if (readers == 0)  
        signal(writeOk);  
}
```

## Operaciones de Escritura

```
public void startWrite() {  
    while (writer || readers >  
        0)  
        wait(writeOk);  
    writer = true;  
}
```

```
public void endWrite() {  
    writer = false;  
    signal(writeOk);  
    signal(readOk);  
}
```

**Nota:** El `signal(readOk)` en `startRead` genera un efecto dominó que despierta a todos los lectores acumulados en cadena.

# Escritores: Control de Prioridad a Lectores

## Finalización del Escritor:

- El escritor que abandona el monitor evalúa el estado de las colas (*empty*).
- Si hay lectores esperando en readOk, se los despierta para garantizar la **Prioridad de Lectores**.
- Si no hay lector esperando, se emite signal en la cola de escritores writeOk.

```
public void startWrite() {  
    while (writer || readers > 0)  
        wait(writeOk);  
    writer = true;  
}  
  
public void endWrite() {  
    writer = false;  
    if (empty(readOk)) {  
        signal(writeOk);  
    } else {  
        signal(readOk); // Prioridad  
                          Lectores  
    }  
}
```

## Comparación: Semáforos vs. Variables de condición

| Semáforo                                                                           | Variable de Condición                                                                              |
|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| wait puede o no bloquear al proceso (depende del valor).                           | wait <b>siempre</b> bloquea al proceso en ejecución.                                               |
| signal <b>siempre</b> tiene un efecto (incrementa el valor o despierta a alguien). | signal <b>no tiene efecto</b> si la cola de la condición está vacía.                               |
| signal desbloquea a un proceso arbitrario de la cola.                              | signal desbloquea estrictamente al proceso que está a la <b>cabeza de la cola</b> (FIFO).          |
| Un proceso desbloqueado por signal puede reanudar su ejecución de forma inmediata. | Un proceso desbloqueado signal debe esperar a que el proceso señalizador libere o deje el monitor. |

# Monitores en Java: Opción 1

- **Estructura básica:** Una clase común y corriente cuyos métodos se declaran explícitamente como `synchronized`.
- **Gestión de condiciones:**
  - Posee una única **condición anónima** (no declarada de forma explícita).
  - Está asociada intrínsecamente al propio objeto que actúa como lock.
- **Primitivas de sincronización integradas:**
  - `wait()`: Suspende al hilo actual liberando temporalmente el lock.
  - `notify()`: Despierta a un único hilo al azar de la cola de condición.
  - `notifyAll()`: Despierta a todos los hilos bloqueados en la condición.



# Ejemplo de Monitor Simple en Java (Opción 1)

## Características del diseño nativo:

- synchronized garantiza la exclusión mutua al ingresar a los métodos de la instancia.
- wait(), notify() y notifyAll() se invocan directamente (llamada this).
- Cuenta con una única cola de condición implícita por cada objeto.

```
public class SimpleMonitor {  
  
    public synchronized void A()  
        throws InterruptedException {  
        // ... operaciones ...  
        wait(); // Bloquea el hilo  
        // ... continua ...  
    }  
  
    public synchronized void B() {  
        // ... cambia estado ...  
        notifyAll(); // Despierta hilos  
    }  
}
```

## Monitores en Java: Opción 2

- **Mecanismo de exclusión:** Uso de un lock explícito mediante una instancia de la clase `ReentrantLock`.
  - Implementa la interfaz estándar `Lock`.
  - Se controla el acceso manualmente mediante `lock()` y `unlock()`.
- **Gestión de condiciones explícitas:**
  - Permite declarar múltiples variables de condición independientes invocando el método `newCondition()`.
  - Ofrece un control fino de sincronización con sus propias primitivas: `await()`, `signal()` y `signalAll()`.

# Ejemplo de Monitor con Locks Explícitos (Opción 2)

## Ventajas del control explícito:

- ReentrantLock sustituye synchronized.
- Se deben invocar lock() unlock() (unlock dentro de un bloque finally para asegurar la liberación del lock).
- Permite declarar múltiples variables de condición independientes (lock.newCondition()).

```
public class SimpleMonitor {  
    private final Lock lock = new  
        ReentrantLock();  
    private Condition c = lock.  
        newCondition();  
  
    public void A() throws  
        InterruptedException {  
        lock.lock();  
        try {  
            // ... operaciones ...  
            c.await(); // Espera explícita  
        } finally {  
            lock.unlock();  
        }  
    }  
}
```

# Lectores/Escritores: Solución Alternativa en Java

## Estructura del Monitor:

- **readers:** Lectores leyendo.
- **Condición de vaciado:** No quedan lectores activos cuando `readIn == readOut`.

```
public class RW {  
    private final Lock lock = new ReentrantLock();  
    private final Condition condition = lock.newCondition();  
    private int readers = 0;  
    private boolean writer = false;  
  
    // ... las operaciones siguen en las proximas  
    diapositivas
```

# RW Alternativo: Operaciones de Lectura

## Entrada (startRead)

```
public void startRead()
    throws
    InterruptedException {
    lock.lock();
    try {
        while (writer)
            condition.await();
        readers++;
    } finally {
        lock.unlock();
    }
}
```

## Salida (endRead)

```
public void endRead() {
    lock.lock();
    try {
        readers--;
        if (readers == 0)
            condition.signalAll();
    } finally {
        lock.unlock();
    }
}
```

# RW Alternativo: Operaciones de Escritura

## Entrada (startWrite)

```
public void startWrite()
    throws InterruptedException
{
    lock.lock();
    try {
        while (writer)
            condition.await();
        writer = true;

        while (readers != 0)
            condition.await();
    } finally {
        lock.unlock();
    }
}
```

## Salida (endWrite)

```
public void endWrite() {
    lock.lock();
    try {
        writer = false;
        condition.signalAll();
    } finally {
        lock.unlock();
    }
}
```